

# Backend-Optimierung und Nachmessung

Y-Dude | Grundlage: Lasttest 15.08.2026 | 250 / 500 / 750 gleichzeitige Nutzer

## Zusammenfassung

Auf Basis des Lasttests wurden drei Maßnahmen umgesetzt: HTTP-/SSR-Caching fuer öffentliche Seiten, Bündelung von sechs persönlichen Sitzungsabfragen in einen Aufruf und längere Gültigkeit fuer Stammdaten. Funktion, Design und Datenlogik blieben unverändert. Ergebnis unter identischen Bedingungen: **Durchsatz +140 bis +164 Prozent, Antwortzeiten rund 60 Prozent niedriger**, weiterhin **0 Fehler und 0 Timeouts** in allen Stufen.

Der zuvor vermutete Deckel von 256 gleichzeitigen Anfragen war **keine Servergrenze**, sondern das Standardlimit des Testwerkzeugs. Mit angehobenem Limit wurden 510 gleichzeitige Anfragen am Server gemessen.

## 1. Vergleich vorher / nachher (identische Bedingungen)

| Nutzer | Messwert           | vorher         | nachher        | Änderung        |
|--------|--------------------|----------------|----------------|-----------------|
| 250    | Anfragen/s         | 329,8          | 789,7          | +139,5 %        |
| 250    | Antwortzeit Mittel | 755 ms         | 315 ms         | -58,3 %         |
| 250    | p50 / p90          | 838 / 1015 ms  | 313 / 613 ms   | -62,6 / -39,6 % |
| 250    | p95 / p99          | 1423 / 3642 ms | 664 / 859 ms   | -53,3 / -76,4 % |
| 500    | Anfragen/s         | 329,3          | 868,7          | +163,8 %        |
| 500    | Antwortzeit Mittel | 1498 ms        | 573 ms         | -61,7 %         |
| 500    | p50 / p90          | 1347 / 2104 ms | 571 / 837 ms   | -57,6 / -60,2 % |
| 500    | p95 / p99          | 2301 / 2892 ms | 909 / 1051 ms  | -60,5 / -63,7 % |
| 750    | Anfragen/s         | 316,2          | 790,0          | +149,8 %        |
| 750    | Antwortzeit Mittel | 2328 ms        | 942 ms         | -59,5 %         |
| 750    | p50 / p90          | 2202 / 2981 ms | 967 / 1204 ms  | -56,1 / -59,6 % |
| 750    | p95 / p99          | 3115 / 3797 ms | 1309 / 1478 ms | -58,0 / -61,1 % |

Alle Stufen vorher und nachher: 0 Fehler, 0 Timeouts, 0 Datenbankfehler.

## 2. Systemwerte nachher

| Nutzer | Anfragen | gleichzeitig | Serverzeit Mittel | Event-Loop-Lag | RAM (RSS) | CPU (45 s) | Cache-Treff er | DB-Abfrag en |
|--------|----------|--------------|-------------------|----------------|-----------|------------|----------------|--------------|
| 250    | 35 827   | 192          | 179 ms            | 1 ms           | 2344 MB   | 51,2 s     | 14 365         | 3            |
| 500    | 39 467   | 192          | 157 ms            | 2 ms           | 2339 MB   | 48,3 s     | 13 881         | 3            |
| 750    | 36 301   | 198          | 148 ms            | 2 ms           | 2344 MB   | 48,2 s     | 14 042         | 3            |

Cache-Trefferquote des Server-Caches weiterhin ueber 99,9 Prozent. Nur drei Datenbankabfragen je Laststufe, RAM stabil, Event-Loop praktisch ohne Verzögerung: die Datenbank war zu keinem Zeitpunkt der Engpass.

## 3. Die Ursache des 256er-Deckels

Der Lasttest nutzt Bun-fetch. Bun begrenzt gleichzeitige HTTP-Anfragen pro Prozess standardmäßig auf 256. Der Wert war damit eine Grenze der Messung. Gegenprobe mit Burst-Anfragen, gemessen am Server:

| Client-Limit   | Burst | gleichzeitig am Server |
|----------------|-------|------------------------|
| 256 (Standard) | 800   | 154                    |
| 2048           | 800   | 201                    |
| 2048           | 1200  | 510                    |

Es existiert keine Server-Konfiguration mit einem Limit von 256 - weder im Server-Einstieg, noch im Build, noch in einer eigenen Warteschlange. Es wurde deshalb kein Limit erhöht, weil keines vorhanden war. Oberhalb von etwa 250 Nutzern begrenzt die Rechenzeit je Anfrage (serverseitiges Rendern), nicht die Anzahl gleichzeitiger Verbindungen. Genau dort setzten die Optimierungen an.

## 4. Umgesetzte Optimierungen

| Maßnahme                                | Was geändert wurde                                                                                                                                                           | Wirkung                                                        |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| HTTP-/SSR-Cache fuer öffentliche Seiten | Startseite (60 s), Login und Passwort-Reset (300 s), AGB, Datenschutz, Impressum, Richtlinien (3600 s) erhalten<br>Cache-Kopfzeilen und werden kurz in der Instanz gehalten. | Startseite von rund 700 ms auf 42 ms; größter Einzeleffekt     |
| Sitzungsabfragen gebündelt              | Neue Lesefunktion liefert Interessen, Standort, Sprache, Gefolgte, gelernte Gewichte, gefolgte Hashtags und Verbindungen in einem Aufruf.                                    | 6 Datenbankwege werden zu 1; Fallback auf den alten Weg bleibt |
| Stammdaten länger gültig                | Interest-Engine-Konfiguration 300 auf 900 s, Kategorien 600 auf 3600 s.                                                                                                      | weniger Wiederholabfragen ohne inhaltliche Änderung            |

## 5. Datenschutz und Sicherheit des Caches

Gecacht wird ausschließlich bei GET, Status 200 und **ohne jeden Cookie und ohne Authorization-Kopfzeile**. Sobald eine Anmeldung mitkommt, wird die Seite frisch erzeugt, nicht gespeichert und ohne Cache-Kopfzeile ausgeliefert (nachgemessen). Jede gecachte Antwort traegt Vary: Cookie, Authorization; Antworten, die ein Cookie setzen, werden nie gespeichert. Feed, Profile, Messenger, Beiträge, Administration und alle Schnittstellen bleiben unverändert dynamisch. Die gebündelte Lesefunktion liest nur Daten des angemeldeten Nutzers und ist nur fuer angemeldete Rollen ausführbar. Es ist damit ausgeschlossen, dass ein Nutzer Daten eines anderen erhält.

## 6. Instanzgroee und Grenzen der Plattform

| Punkt                         | Bewertung                                                                                                                         |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Begrenzender Faktor           | Rechenzeit fuer das serverseitige Rendern (CPU). Nicht Parallelität, nicht Datenbank, nicht Speicher.                             |
| Nutzen einer größeren Instanz | Mehr CPU bedeutet mehr Seitenaufbauten je Sekunde. Sinnvoll erst bei dauerhaft mehr als 800 Anfragen/s dynamischer Inhalte.       |
| In Lovable möglich            | Größe der Cloud-Instanz erhöhen (Backend, Advanced settings, Upgrade instance).                                                   |
| In Lovable nicht einstellbar  | Anzahl der Worker, Parallelitäts- und Keep-Alive-Grenzen, mehrere Instanzen, Lastverteilung. Diese Werte verwaltet die Plattform. |
| Kosten und Risiko             | Größere Instanz erhöht die Cloud-Nutzung; Umstellung dauert einige Minuten. Derzeit nicht erforderlich.                           |

## 7. Zusatzlauf mit angehobenem Client-Limit (4096)

| Nutzer | Anfragen/s | Mittel | p95     | p99       | gleichzeitig | Fehler / Timeouts |
|--------|------------|--------|---------|-----------|--------------|-------------------|
| 250    | 827,6      | 301 ms | 651 ms  | 732 ms    | 198          | 0 / 0             |
| 500    | 900,7      | 553 ms | 968 ms  | 1100 ms   | 241          | 0 / 0             |
| 750    | 876,0      | 796 ms | 1277 ms | 11 026 ms | 248          | 116 / 116         |

Ehrliche Einordnung: bei 750 Nutzern und entfesseltem Client traten 116 Timeouts auf (0,3 Prozent) und p99 brach ein. Mehr Parallelität bringt oberhalb von rund 900 Anfragen/s keinen Gewinn mehr, sondern erzeugt eine Warteschlange. Der höchste sauber gemessene Durchsatz liegt bei 900,7 Anfragen/s.

## 8. Fazit und nächste Schritte

Stabiler Lastbereich neu: bis 750 gleichzeitige Nutzer fehlerfrei bei 790 bis 870 Anfragen/s und p95 unter 1,4 Sekunden. Der Engpass ist jetzt die Rechenzeit dynamischer Ansichten. Sinnvolle nächste Schritte in dieser Reihenfolge: dynamische Feed-Ansichten weiter entlasten, Bilder und Medien ueber den CDN-Cache ausliefern, danach - und erst bei Bedarf - die Cloud-Instanz vergrößern. Externe Infrastruktur wird erst bei dauerhaft mehr als 1000 Anfragen/s dynamischer Inhalte oder mehreren Regionen relevant.